

isaacsim.terrains

Contents

- Terrain importer
- Terrain generator
- Height fields
- Trimesh terrains
- Utilities

Sub-package with utilities for creating terrains procedurally.

There are two main components in this package:

- `TerrainGenerator`: This class procedurally generates terrains based on the passed sub-terrain configuration. It creates a `trimesh` mesh object and contains the origins of each generated sub-terrain.
- `TerrainImporter`: This class mainly deals with importing terrains from different possible sources and adding them to the simulator as a prim object. The following functions are available for importing terrains:
 - `TerrainImporter.import_ground_plane()`: spawn a grid plane which is default in Isaac Sim.
 - `TerrainImporter.import_mesh()`: spawn a prim from a `trimesh` object.
 - `TerrainImporter.import_usd()`: spawn a prim as reference to input USD file.

Classes

<code>TerrainImporter</code>	A class to handle terrain meshes and import them into the simulator.
<code>TerrainImporterCfg</code>	Configuration for the terrain manager.
<code>TerrainGenerator</code>	Terrain generator to handle different terrain generation functions.
<code>TerrainGeneratorCfg</code>	Configuration for the terrain generator.
<code>SubTerrainBaseCfg</code>	Base class for terrain configurations.

Terrain importer

`class isaacsim.terrains.TerrainImporter` [\[source\]](#)

Bases: `object`

A class to handle terrain meshes and import them into the simulator.

We assume that a terrain mesh comprises of sub-terrains that are arranged in a grid with rows `num_rows` and columns `num_cols`. The terrain origins are the positions of the sub-terrains where the robot should be spawned.

Based on the configuration, the terrain importer handles computing the environment origins from the sub-terrain origins. In a typical setup, the number of sub-terrains ($num_rows \times num_cols$) is smaller than the number of environments (num_envs). In this case, the environment origins are computed by sampling the sub-terrain origins.

If a curriculum is used, it is possible to update the environment origins to terrain origins that correspond to a harder difficulty. This is done by calling `update_terrain_levels()`. The idea comes from game-based curriculum. For example, in a game, the player starts with easy levels and progresses to harder levels.

Methods:

<code>__init__</code> (cfg)	Initialize the terrain importer.
<code>set_debug_vis</code> (debug_vis)	Set the debug visualization of the terrain importer.
<code>import_ground_plane</code> (name[, size])	Add a plane to the terrain importer.
<code>import_mesh</code> (name, mesh)	Import a mesh into the simulator.
<code>import_usd</code> (name, usd_path)	Import a mesh from a USD file.
<code>configure_env_origins</code> ([origins])	Configure the origins of the environments based on the added terrain.
<code>update_env_origins</code> (env_ids, move_up, move_down)	Update the environment origins based on the terrain levels.

Attributes:

<code>terrain_prim_paths</code>	A list containing the USD prim paths to the imported terrains.
<code>terrain_origins</code>	The origins of the sub-terrains in the added terrain mesh.
<code>env_origins</code>	The origins of the environments.
<code>has_debug_vis_implementation</code>	Whether the terrain importer has a debug visualization implemented.
<code>flat_patches</code>	A dictionary containing the sampled valid (flat) patches for the terrain.
<code>terrain_names</code>	A list of names of the imported terrains.
<code>warp_meshes</code>	A dictionary containing the terrain's names and their warp meshes.
<code>meshes</code>	A dictionary containing the terrain's names and their tri-meshes.

`__init__(cfg: TerrainImporterCfg)`

[\[source\]](#)

Initialize the terrain importer.

Parameters:

cfg – The configuration for the terrain importer.

Raises:

- **ValueError** – If input terrain type is not supported.
- **ValueError** – If terrain type is 'generator' and no configuration provided for `terrain_generator`.
- **ValueError** – If terrain type is 'usd' and no configuration provided for `usd_path`.
- **ValueError** – If terrain type is 'usd' or 'plane' and no configuration provided for `env_spacing`.

`terrain_prim_paths: List[str]`

A list containing the USD prim paths to the imported terrains.

`terrain_origins: torch.Tensor | None`

The origins of the sub-terrains in the added terrain mesh. Shape is (num_rows, num_cols, 3).

If terrain origins is not None, the environment origins are computed based on the terrain origins. Otherwise, the environment origins are computed based on the grid spacing.

`env_origins: torch.Tensor`

The origins of the environments. Shape is (num_envs, 3).

`property has_debug_vis_implementation: bool`

Whether the terrain importer has a debug visualization implemented.

This always returns True.

`property flat_patches: dict[str, torch.Tensor]`

A dictionary containing the sampled valid (flat) patches for the terrain.

This is only available if the terrain type is 'generator'. For other terrain types, this feature is not available and the function returns an empty dictionary.

Please refer to the `TerrainGenerator.flat_patches` for more information.

`property terrain_names: List[str]`

A list of names of the imported terrains.

```
set_debug_vis(debug_vis: bool) → bool
```

[\[source\]](#)

Set the debug visualization of the terrain importer.

Parameters:

debug_vis – Whether to visualize the terrain origins.

Returns:

Whether the debug visualization was successfully set. False if the terrain importer does not support debug visualization.

Raises:

RuntimeError – If terrain origins are not configured.

```
import_ground_plane(name: str, size: tuple[float, float] = (2000000.0, 2000000.0))
```

[\[source\]](#)

Add a plane to the terrain importer.

Parameters:

- **name** – The name of the imported terrain. This name is used to create the USD prim corresponding to the terrain.
- **size** – The size of the plane. Defaults to (2.0e6, 2.0e6).

Raises:

ValueError – If a terrain with the same name already exists.

```
import_mesh(name: str, mesh: trimesh.Trimesh)
```

[\[source\]](#)

Import a mesh into the simulator.

The mesh is imported into the simulator under the prim path `cfg.prim_path/{key}`. The created path contains the mesh as a `pxr.UsdGeom` instance along with visual or physics material prims.

Parameters:

- **name** – The name of the imported terrain. This name is used to create the USD prim corresponding to the terrain.
- **mesh** – The mesh to import.

Raises:

ValueError – If a terrain with the same name already exists.

```
import_usd(name: str, usd_path: str)
```

[\[source\]](#)

Import a mesh from a USD file.

This function imports a USD file into the simulator as a terrain. It parses the USD file and stores the mesh under the prim path `cfg.prim_path/{key}`. If multiple meshes are present in the USD file, only the first

mesh is imported.

The function does not apply any material properties to the mesh. The material properties should be defined in the USD file.

Parameters:

- **name** – The name of the imported terrain. This name is used to create the USD prim corresponding to the terrain.
- **usd_path** – The path to the USD file.

Raises:

ValueError – If a terrain with the same name already exists.

configure_env_origins(*origins: np.ndarray | torch.Tensor | None = None*) [\[source\]](#)

Configure the origins of the environments based on the added terrain.

Parameters:

origins – The origins of the sub-terrains. Shape is (num_rows, num_cols, 3).

update_env_origins(*env_ids: torch.Tensor, move_up: torch.Tensor, move_down: torch.Tensor*) [\[source\]](#)

Update the environment origins based on the terrain levels.

property **warp_meshes**

A dictionary containing the terrain's names and their warp meshes.

! Deprecated since version v2.1.0: The *warp_meshes* attribute is deprecated. It is no longer stored inside the class.

property **meshes: dict[str, trimesh.Trimesh]**

A dictionary containing the terrain's names and their tri-meshes.

! Deprecated since version v2.1.0: The *meshes* attribute is deprecated. It is no longer stored inside the class.

class **isaacclab.terrains.TerrainImporterCfg** [\[source\]](#)

Configuration for the terrain manager.

Attributes:

<code>collision_group</code>	The collision group of the terrain.
<code>prim_path</code>	The absolute path of the USD terrain prim.
<code>num_envs</code>	The number of environment origins to consider.
<code>terrain_type</code>	The type of terrain to generate.
<code>terrain_generator</code>	The terrain generator configuration.
<code>usd_path</code>	The path to the USD file containing the terrain.
<code>env_spacing</code>	The spacing between environment origins when defined in a grid.
<code>use_terrain_origins</code>	Whether to set the environment origins based on the terrain origins or in a grid according to <code>env_spacing</code> .
<code>visual_material</code>	The visual material of the terrain.
<code>physics_material</code>	The physics material of the terrain.
<code>max_init_terrain_level</code>	The maximum initial terrain level for defining environment origins.
<code>debug_vis</code>	Whether to enable visualization of terrain origins for the terrain.

`collision_group: int`

The collision group of the terrain. Defaults to -1.

`prim_path: str`

The absolute path of the USD terrain prim.

All sub-terrains are imported relative to this prim path.

`num_envs: int`

The number of environment origins to consider. Defaults to 1.

In case, the `InteractiveSceneCfg` is used, this parameter gets overridden by `isaac.lab.scene.InteractiveSceneCfg.num_envs` attribute.

`terrain_type: Literal['generator', 'plane', 'usd']`

The type of terrain to generate. Defaults to “generator”.

Available options are “plane”, “usd”, and “generator”.

`terrain_generator: TerrainGeneratorCfg | None`

The terrain generator configuration.

Only used if `terrain_type` is set to “generator”.

USD path: `str` | `None`

The path to the USD file containing the terrain.

Only used if `terrain_type` is set to “USD”.

env_spacing: `float` | `None`

The spacing between environment origins when defined in a grid. Defaults to None.

Note

This parameter is used only when the `terrain_type` is “plane” or “USD” or if `use_terrain_origins` is False.

use_terrain_origins: `bool`

Whether to set the environment origins based on the terrain origins or in a grid according to `env_spacing`. Defaults to True.

Note

This parameter is used only when the `terrain_type` is “generator”.

visual_material: `sim_utils.VisualMaterialCfg` | `None`

The visual material of the terrain. Defaults to a dark gray color material.

This parameter is used for both the “generator” and “plane” terrains.

- If the `terrain_type` is “generator”, then the material is created at the path `{prim_path}/visualMaterial` and applied to all the sub-terrains.
- If the `terrain_type` is “plane”, then the diffuse color of the material is set to the grid color of the imported ground plane.

physics_material: `sim_utils.RigidBodyMaterialCfg`

The physics material of the terrain. Defaults to a default physics material.

The material is created at the path: `{prim_path}/physicsMaterial`.

Note

This parameter is used only when the `terrain_type` is “generator” or “plane”.

`max_init_terrain_level: int | None`

The maximum initial terrain level for defining environment origins. Defaults to None.

The terrain levels are specified by the number of rows in the grid arrangement of sub-terrains. If None, then the initial terrain level is set to the maximum terrain level available (`num_rows - 1`).

Note

This parameter is used only when sub-terrain origins are defined.

`debug_vis: bool`

Whether to enable visualization of terrain origins for the terrain. Defaults to False.

Terrain generator

`class isaacsim.terrains.TerrainGenerator`

[\[source\]](#)

Terrain generator to handle different terrain generation functions.

The terrains are represented as meshes. These are obtained either from height fields or by using the `trimesh` library. The height field representation is more flexible, but it is less computationally and memory efficient than the trimesh representation.

All terrain generation functions take in the argument `difficulty` which determines the complexity of the terrain. The difficulty is a number between 0 and 1, where 0 is the easiest and 1 is the hardest. In most cases, the difficulty is used for linear interpolation between different terrain parameters. For example, in a pyramid stairs terrain the step height is interpolated between the specified minimum and maximum step height.

Each sub-terrain has a corresponding configuration class that can be used to specify the parameters of the terrain. The configuration classes are inherited from the `SubTerrainBaseCfg` class which contains the common parameters for all terrains.

If a curriculum is used, the terrains are generated based on their difficulty parameter. The difficulty is varied linearly over the number of rows (i.e. along x) with a small random value added to the difficulty to ensure that the columns with the same sub-terrain type are not exactly the same. The difficulty parameter for a sub-terrain at a given row is calculated as:

$$\text{difficulty} = \frac{\text{row_id} + \eta}{\text{num_rows}} \times (\text{upper} - \text{lower}) + \text{lower}$$

where $\eta \sim \mathcal{U}(0, 1)$ is a random perturbation to the difficulty, and $(\text{lower}, \text{upper})$ is the range of the difficulty parameter, specified using the `difficulty_range` parameter.

If a curriculum is not used, the terrains are generated randomly. In this case, the difficulty parameter is randomly sampled from the specified range, given by the `difficulty_range` parameter:

$$\text{difficulty} \sim \mathcal{U}(\text{lower}, \text{upper})$$

If the `flat_patch_sampling` is specified for a sub-terrain, flat patches are sampled on the terrain. These can be used for spawning robots, targets, etc. The sampled patches are stored in the `flat_patches` dictionary. The key specifies the intention of the flat patches and the value is a tensor containing the flat patches for each sub-terrain.

If the flag `use_cache` is set to True, the terrains are cached based on their sub-terrain configurations. This means that if the same sub-terrain configuration is used multiple times, the terrain is only generated once and then reused. This is useful when generating complex sub-terrains that take a long time to generate.

! Attention

The terrain generation has its own seed parameter. This is set using the `TerrainGeneratorCfg.seed` parameter. If the seed is not set and the caching is disabled, the terrain generation may not be completely reproducible.

Methods:

<code>__init__</code> (cfg[, device])	Initialize the terrain generator.
---------------------------------------	-----------------------------------

Attributes:

<code>flat_patches</code>	A dictionary of sampled valid (flat) patches for each sub-terrain.
<code>terrain_meshes</code>	List of trimesh.Trimesh objects for all the generated sub-terrains.
<code>terrain_origins</code>	The origin of each sub-terrain.
<code>terrain_mesh</code>	A single trimesh.Trimesh object for all the generated sub-terrains.

`__init__(cfg: TerrainGeneratorCfg, device: str = 'cpu')` [\[source\]](#)

Initialize the terrain generator.

Parameters:

- **cfg** – Configuration for the terrain generator.
- **device** – The device to use for the flat patches tensor.

flat_patches: *dict[str, torch.Tensor]*

A dictionary of sampled valid (flat) patches for each sub-terrain.

The dictionary keys are the names of the flat patch sampling configurations. This maps to a tensor containing the flat patches for each sub-terrain. The shape of the tensor is (num_rows, num_cols, num_patches, 3).

For instance, the key “root_spawn” maps to a tensor containing the flat patches for spawning an asset. Similarly, the key “target_spawn” maps to a tensor containing the flat patches for setting targets.

terrain_meshes: *list[trimesh.Trimesh]*

List of trimesh.Trimesh objects for all the generated sub-terrains.

terrain_origins: *ndarray*

The origin of each sub-terrain. Shape is (num_rows, num_cols, 3).

terrain_mesh: *trimesh.Trimesh*

A single trimesh.Trimesh object for all the generated sub-terrains.

class isaacsim.terrains.TerrainGeneratorCfg

[\[source\]](#)

Configuration for the terrain generator.

Classes:

`class_type`

alias of `TerrainGenerator`

Attributes:

<code>seed</code>	The seed for the random number generator.
<code>curriculum</code>	Whether to use the curriculum mode.
<code>size</code>	The width (along x) and length (along y) of each sub-terrain (in m).
<code>border_width</code>	The width of the border around the terrain (in m).
<code>border_height</code>	The height of the border around the terrain (in m).
<code>num_rows</code>	Number of rows of sub-terrains to generate.
<code>num_cols</code>	Number of columns of sub-terrains to generate.
<code>color_scheme</code>	Color scheme to use for the terrain.
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>sub_terrains</code>	Dictionary of sub-terrain configurations.
<code>difficulty_range</code>	The range of difficulty values for the sub-terrains.
<code>use_cache</code>	Whether to load the sub-terrain from cache if it exists.
<code>cache_dir</code>	The directory where the terrain cache is stored.

class_type

alias of `TerrainGenerator`

seed: *int* | *None*

The seed for the random number generator. Defaults to None, in which case the seed from the current NumPy's random state is used.

When the seed is set, the random number generator is initialized with the given seed. This ensures that the generated terrains are deterministic across different runs. If the seed is not set, the seed from the current NumPy's random state is used. This assumes that the seed is set elsewhere in the code.

curriculum: *bool*

Whether to use the curriculum mode. Defaults to False.

If True, the terrains are generated based on their difficulty parameter. Otherwise, they are randomly generated.

size: *tuple[float, float]*

The width (along x) and length (along y) of each sub-terrain (in m).

Note

This value is passed on to all the sub-terrain configurations.

border_width: *float*

The width of the border around the terrain (in m). Defaults to 0.0.

border_height: *float*

The height of the border around the terrain (in m). Defaults to 1.0.

Note

The default border extends below the ground. If you want to make the border above the ground, choose a negative value.

num_rows: *int*

Number of rows of sub-terrains to generate. Defaults to 1.

num_cols: *int*

Number of columns of sub-terrains to generate. Defaults to 1.

color_scheme: *Literal['height', 'random', 'none']*

Color scheme to use for the terrain. Defaults to “none”.

The available color schemes are:

- “height”: Color based on the height of the terrain.
- “random”: Random color scheme.
- “none”: No color scheme.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

This value is passed on to all the height field sub-terrain configurations.

vertical_scale: float

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

This value is passed on to all the height field sub-terrain configurations.

slope_threshold: float | None

The slope threshold above which surfaces are made vertical. Defaults to 0.75.

If None no correction is applied.

This value is passed on to all the height field sub-terrain configurations.

sub_terrains: dict[str, SubTerrainBaseCfg]

Dictionary of sub-terrain configurations.

The keys correspond to the name of the sub-terrain configuration and the values are the corresponding configurations.

difficulty_range: tuple[float, float]

The range of difficulty values for the sub-terrains. Defaults to (0.0, 1.0).

If curriculum is enabled, the terrains will be generated based on this range in ascending order of difficulty. Otherwise, the terrains will be generated based on this range in a random order.

use_cache: bool

Whether to load the sub-terrain from cache if it exists. Defaults to False.

If enabled, the generated terrains are stored in the cache directory. When generating terrains, the cache is checked to see if the terrain already exists. If it does, the terrain is loaded from the cache. Otherwise, the terrain is generated and stored in the cache. Caching can be used to speed up terrain generation.

cache_dir: str

The directory where the terrain cache is stored. Defaults to "/tmp/isaacsim/terrains".

class isaacsim.terrains.SubTerrainBaseCfg[\[source\]](#)

Base class for terrain configurations.

All the sub-terrain configurations must inherit from this class.

The `size` attribute is the size of the generated sub-terrain. Based on this, the terrain must extend from (0, 0)

to `(size[0], size[1])`.

Attributes:

<code>function</code>	Function to generate the terrain.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

function: `Callable[[float, SubTerrainBaseCfg], tuple[list[trimesh.Trimesh], ndarray]]`

Function to generate the terrain.

This function must take as input the terrain difficulty and the configuration parameters and return a tuple with a list of `trimesh` mesh objects and the terrain origin.

proportion: `float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: `tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: `dict[str, FlatPatchSamplingCfg] | None`

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Height fields

This sub-module provides utilities to create different terrains as height fields (HF).

Height fields are a 2.5D terrain representation that is used in robotics to obtain the height of the terrain at a given point. This is useful for controls and planning algorithms.

Each terrain is represented as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively. The height of the terrain at a given point is obtained by indexing the array with the corresponding x and y coordinates.

⚠ Caution

When working with height field terrains, it is important to remember that the terrain is generated from a discretized 3D representation. This means that the height of the terrain at a given point is only an approximation of the real height of the terrain at that point. The discretization error is proportional to the size of the discretization cells. Therefore, it is important to choose a discretization size that is small enough for the application. A larger discretization size will result in a faster simulation, but the terrain will be less accurate.

All sub-terrains must inherit from the `HfTerrainBaseCfg` class which contains the common parameters for all terrains generated from height fields.

`class isaacsim.terrains.height_field.hf_terrains_cfg.HfTerrainBaseCfg` [\[source\]](#)

Bases: `SubTerrainBaseCfg`

The base configuration for height field terrains.

Attributes:

<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`border_width: float`

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float* | *None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple*[*float*, *float*]

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict*[*str*, *FlatPatchSamplingCfg*] | *None*

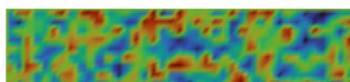
Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

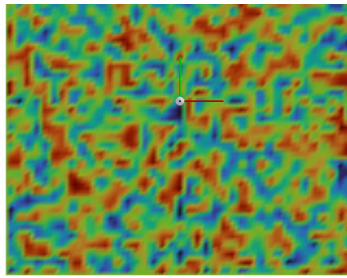
The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Random Uniform Terrain

`isaacsim.terrains.height_field.hf_terrains.random_uniform_terrain(difficulty: float, cfg: hf_terrains_cfg.HfRandomUniformTerrainCfg) → np.ndarray` [\[source\]](#)

Generate a terrain with height sampled uniformly from a specified range.



**Note**

The `difficulty` parameter is ignored for this terrain.

Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

The height field of the terrain as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively.

Raises:

ValueError – When the downsampled scale is smaller than the horizontal scale.

```
class isaacsim.terrains.height_field.hf_terrains_cfg.HfRandomUniformTerrainCfg
```

Bases: `HfTerrainBaseCfg`

[\[source\]](#)

Configuration for a random uniform height field terrain.

Attributes:

<code>noise_range</code>	The minimum and maximum height noise (i.e. along z) of the terrain (in m).
<code>noise_step</code>	The minimum height (in m) change between two points.
<code>downsampled_scale</code>	The distance between two randomly sampled points on the terrain.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.

`noise_range`: *tuple[float, float]*

The minimum and maximum height noise (i.e. along z) of the terrain (in m).

`noise_step`: *float*

The minimum height (in m) change between two points.

`downsampled_scale`: *float | None*

The distance between two randomly sampled points on the terrain. Defaults to None, in which case the `horizontal scale` is used.

The heights are sampled at this resolution and interpolation is performed for intermediate points. This must be larger than or equal to the `horizontal scale`.

`proportion`: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size`: *tuple[float, float]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal_scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float | None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

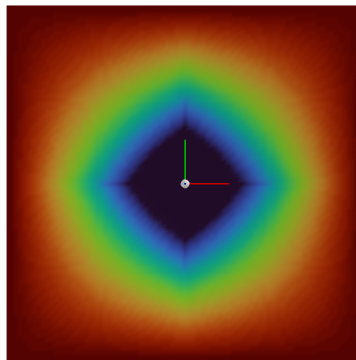
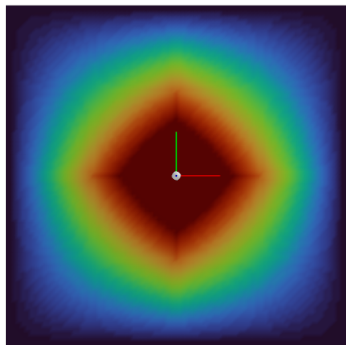
Pyramid Sloped Terrain

`isaacsim.terrains.height_field.hf_terrains.pyramid_sloped_terrain(difficulty: float, cfg: hf_terrains_cfg.HfPyramidSlopedTerrainCfg) → np.ndarray` [\[source\]](#)

Generate a terrain with a truncated pyramid structure.

The terrain is a pyramid-shaped sloped surface with a slope of `slope` that trims into a flat platform at the center. The slope is defined as the ratio of the height change along the x axis to the width along the x axis. For example, a slope of 1.0 means that the height changes by 1 unit for every 1 unit of width.

If the `cfg.inverted` flag is set to `True`, the terrain is inverted such that the platform is at the bottom.

**Parameters:**

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

The height field of the terrain as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively.

class isaacsim.terrains.height_field.hf_terrains_cfg.HfPyramidSlopedTerrainCfg

Bases: [HfTerrainBaseCfg](#)

[\[source\]](#)

Configuration for a pyramid sloped height field terrain.

Attributes:

slope_range	The slope of the terrain (in radians).
platform_width	The width of the square platform at the center of the terrain.
inverted	Whether the pyramid is inverted.
proportion	Proportion of the terrain to generate.
size	The width (along x) and length (along y) of the terrain (in m).
flat_patch_sampling	Dictionary of configurations for sampling flat patches on the sub-terrain.
border_width	The width of the border/padding around the terrain (in m).
horizontal_scale	The discretization of the terrain along the x and y axes (in m).
vertical_scale	The discretization of the terrain along the z axis (in m).
slope_threshold	The slope threshold above which surfaces are made vertical.

slope_range: *tuple[float, float]*

The slope of the terrain (in radians).

platform_width: float

The width of the square platform at the center of the terrain. Defaults to 1.0.

inverted: bool

Whether the pyramid is inverted. Defaults to False.

If True, the terrain is inverted such that the platform is at the bottom and the slopes are upwards.

proportion: float

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: tuple[float, float]

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: dict[str, FlatPatchSamplingCfg] | None

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: float

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal_scale`.

horizontal_scale: float

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float* | *None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

class

`isaacsim.terrains.height_field.hf_terrains_cfg.HfInvertedPyramidSlopedTerrainCfg`

Bases: `HfPyramidSlopedTerrainCfg`

[\[source\]](#)

Configuration for an inverted pyramid sloped height field terrain.

Note

This is a subclass of `HfPyramidSlopedTerrainCfg` with `inverted` set to True. We make it as a separate class to make it easier to distinguish between the two and match the naming convention of the other terrains.

Attributes:

<code>inverted</code>	Whether the pyramid is inverted.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>slope_range</code>	The slope of the terrain (in radians).
<code>platform_width</code>	The width of the square platform at the center of the terrain.

inverted: *bool*

Whether the pyramid is inverted. Defaults to False.

If True, the terrain is inverted such that the platform is at the bottom and the slopes are upwards.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[*float*, *float*]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacclab.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float* | *None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

slope_range: *tuple[*float*, *float*]*

The slope of the terrain (in radians).

platform_width: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

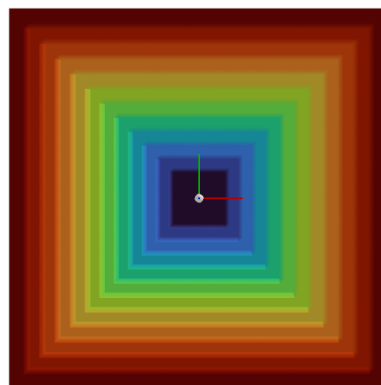
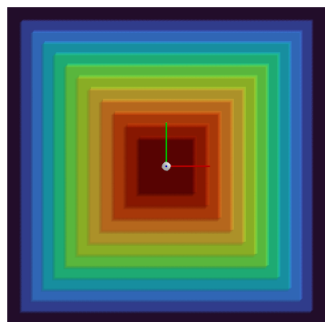
Pyramid Stairs Terrain

`isaacsim.terrains.height_field.hf_terrains.pyramid_stairs_terrain(difficulty: float, cfg: hf_terrains_cfg.HfPyramidStairsTerrainCfg) → np.ndarray` [\[source\]](#)

Generate a terrain with a pyramid stair pattern.

The terrain is a pyramid stair pattern which trims to a flat platform at the center of the terrain.

If the `cfg.inverted` flag is set to `True`, the terrain is inverted such that the platform is at the bottom.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

The height field of the terrain as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively.

`class isaacsim.terrains.height_field.hf_terrains_cfg.HfPyramidStairsTerrainCfg` [\[source\]](#)

Bases: `HfTerrainBaseCfg`

Configuration for a pyramid stairs height field terrain.

Attributes:

<code>step_height_range</code>	The minimum and maximum height of the steps (in m).
<code>step_width</code>	The width of the steps (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>inverted</code>	Whether the pyramid stairs is inverted.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.

`step_height_range`: *tuple[float, float]*

The minimum and maximum height of the steps (in m).

`step_width`: *float*

The width of the steps (in m).

`platform_width`: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

`inverted`: *bool*

Whether the pyramid stairs is inverted. Defaults to False.

If True, the terrain is inverted such that the platform is at the bottom and the stairs are upwards.

`proportion`: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[float, float]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal_scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float | None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

class

`isaacsim.terrains.height_field.hf_terrains_cfg.HfInvertedPyramidStairsTerrainCfg`

Bases: `HfPyramidStairsTerrainCfg`

[\[source\]](#)

Configuration for an inverted pyramid stairs height field terrain.

Note

This is a subclass of `HfPyramidStairsTerrainCfg` with `inverted` set to True. We make it as a separate class to make it easier to distinguish between the two and match the naming convention of the other terrains.

Attributes:

<code>inverted</code>	Whether the pyramid stairs is inverted.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>step_height_range</code>	The minimum and maximum height of the steps (in m).
<code>step_width</code>	The width of the steps (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.

`inverted: bool`

Whether the pyramid stairs is inverted. Defaults to False.

If True, the terrain is inverted such that the platform is at the bottom and the stairs are upwards.

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size: tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling`: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

`border_width`: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal_scale`.

`horizontal_scale`: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

`vertical_scale`: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

`slope_threshold`: *float | None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

`step_height_range`: *tuple[float, float]*

The minimum and maximum height of the steps (in m).

`step_width`: *float*

The width of the steps (in m).

`platform_width`: *float*

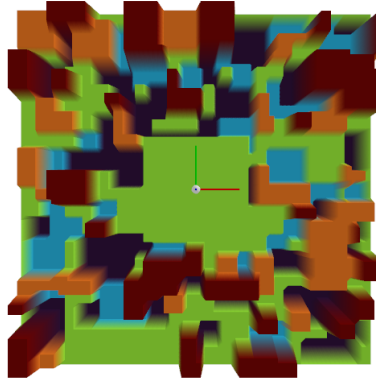
The width of the square platform at the center of the terrain. Defaults to 1.0.

Discrete Obstacles Terrain

```
isaacsim.terrains.height_field.hf_terrains.discrete_obstacles_terrain(difficulty:  
float, cfg: hf_terrains_cfg.HfDiscreteObstaclesTerrainCfg) → np.ndarray \[source\]
```

Generate a terrain with randomly generated obstacles as pillars with positive and negative heights.

The terrain is a flat platform at the center of the terrain with randomly generated obstacles as pillars with positive and negative height. The obstacles are randomly generated cuboids with a random width and height. They are placed randomly on the terrain with a minimum distance of `cfg.platform_width` from the center of the terrain.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

The height field of the terrain as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively.

```
class isaacsim.terrains.height_field.hf_terrains_cfg.HfDiscreteObstaclesTerrainCfg \[source\]
```

Bases: `HfTerrainBaseCfg`

Configuration for a discrete obstacles height field terrain.

Attributes:

<code>obstacle_height_mode</code>	The mode to use for the obstacle height.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>obstacle_width_range</code>	The minimum and maximum width of the obstacles (in m).
<code>obstacle_height_range</code>	The minimum and maximum height of the obstacles (in m).
<code>num_obstacles</code>	The number of obstacles to generate.
<code>platform_width</code>	The width of the square platform at the center of the terrain.

`obstacle_height_mode`: *str*

The mode to use for the obstacle height. Defaults to “choice”.

The following modes are supported: “choice”, “fixed”.

`proportion`: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size`: *tuple[float, float]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling`: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case

no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal_scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float* | *None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

obstacle_width_range: *tuple[float, float]*

The minimum and maximum width of the obstacles (in m).

obstacle_height_range: *tuple[float, float]*

The minimum and maximum height of the obstacles (in m).

num_obstacles: *int*

The number of obstacles to generate.

platform_width: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

Wave Terrain

`isaacsim.terrains.height_field.hf_terrains.wave_terrain(difficulty: float, cfg:`

`hf_terrains_cfg.HfWaveTerrainCfg`) → `np.ndarray`

[\[source\]](#)

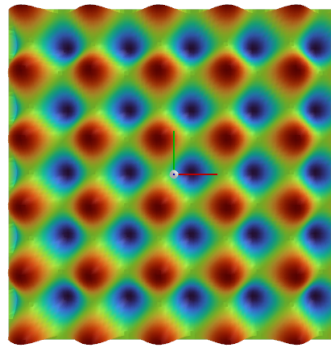
Generate a terrain with a wave pattern.

The terrain is a flat platform at the center of the terrain with a wave pattern. The wave pattern is generated by adding sinusoidal waves based on the number of waves and the amplitude of the waves.

The height of the terrain at a point (x, y) is given by:

$$h(x, y) = A \left(\sin \left(\frac{2\pi x}{\lambda} \right) + \cos \left(\frac{2\pi y}{\lambda} \right) \right)$$

where A is the amplitude of the waves, λ is the wavelength of the waves.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

The height field of the terrain as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively.

Raises:

ValueError – When the number of waves is non-positive.

`class isaacsim.terrains.height_field.hf_terrains_cfg.HfWaveTerrainCfg`

[\[source\]](#)

Bases: `HfTerrainBaseCfg`

Configuration for a wave height field terrain.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>amplitude_range</code>	The minimum and maximum amplitude of the wave (in m).
<code>num_waves</code>	The number of waves to generate.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[*float*, *float*]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float* | *None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

amplitude_range: *tuple*[*float*, *float*]

The minimum and maximum amplitude of the wave (in m).

num_waves: *int*

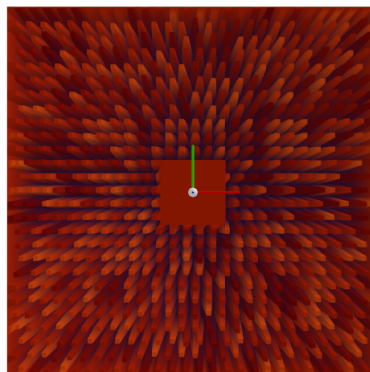
The number of waves to generate. Defaults to 1.

Stepping Stones Terrain

`isaacsim.terrains.height_field.hf_terrains.stepping_stones_terrain(difficulty: float, cfg: hf_terrains_cfg.HfSteppingStonesTerrainCfg) → np.ndarray` [\[source\]](#)

Generate a terrain with a stepping stones pattern.

The terrain is a stepping stones pattern which trims to a flat platform at the center of the terrain.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

The height field of the terrain as a 2D numpy array with discretized heights. The shape of the array is (width, length), where width and length are the number of points along the x and y axis, respectively.

`class isaacsim.terrains.height_field.hf_terrains_cfg.HfSteppingStonesTerrainCfg`

Bases: `HfTerrainBaseCfg`

[\[source\]](#)

Configuration for a stepping stones height field terrain.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border/padding around the terrain (in m).
<code>horizontal_scale</code>	The discretization of the terrain along the x and y axes (in m).
<code>vertical_scale</code>	The discretization of the terrain along the z axis (in m).
<code>slope_threshold</code>	The slope threshold above which surfaces are made vertical.
<code>stone_height_max</code>	The maximum height of the stones (in m).
<code>stone_width_range</code>	The minimum and maximum width of the stones (in m).
<code>stone_distance_range</code>	The minimum and maximum distance between stones (in m).
<code>holes_depth</code>	The depth of the holes (negative obstacles).
<code>platform_width</code>	The width of the square platform at the center of the terrain.

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size: tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by

`isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border/padding around the terrain (in m). Defaults to 0.0.

The border width is subtracted from the `size` of the terrain. If non-zero, it must be greater than or equal to the `horizontal_scale`.

horizontal_scale: *float*

The discretization of the terrain along the x and y axes (in m). Defaults to 0.1.

vertical_scale: *float*

The discretization of the terrain along the z axis (in m). Defaults to 0.005.

slope_threshold: *float | None*

The slope threshold above which surfaces are made vertical. Defaults to None, in which case no correction is applied.

stone_height_max: *float*

The maximum height of the stones (in m).

stone_width_range: *tuple[float, float]*

The minimum and maximum width of the stones (in m).

stone_distance_range: *tuple[float, float]*

The minimum and maximum distance between stones (in m).

holes_depth: *float*

The depth of the holes (negative obstacles). Defaults to -10.0.

platform_width: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

Trimesh terrains

This sub-module provides methods to create different terrains using the `trimesh` library.

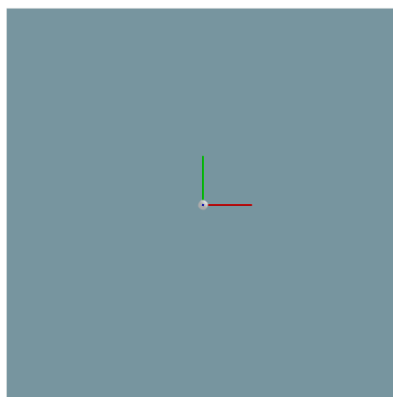
In contrast to the height-field representation, the trimesh representation does not create arbitrarily small triangles. Instead, the terrain is represented as a single tri-mesh primitive. Thus, this representation is more computationally and memory efficient than the height-field representation, but it is not as flexible.

Flat terrain

`isaacsim.terrains.trimesh.mesh_terrains.flat_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshPlaneTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]`

Generate a flat terrain as a plane.

[\[source\]](#)



Note

The `difficulty` parameter is ignored for this terrain.

Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

`class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshPlaneTerrainCfg` [\[source\]](#)

Bases: `SubTerrainBaseCfg`

Configuration for a plane mesh terrain.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[float, float]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by

`isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

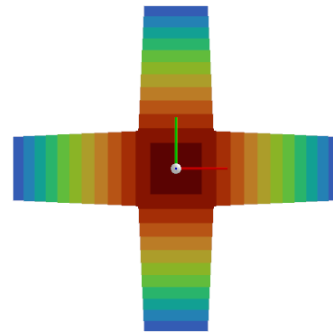
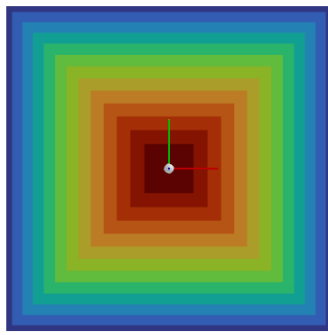
Pyramid terrain

`isaacsim.terrains.trimesh.mesh_terrains.pyramid_stairs_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshPyramidStairsTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]` [\[source\]](#)

Generate a terrain with a pyramid stair pattern.

The terrain is a pyramid stair pattern which trims to a flat platform at the center of the terrain.

If `cfg.holes` is True, the terrain will have pyramid stairs of length or width `cfg.platform_width` (depending on the direction) with no steps in the remaining area. Additionally, no border will be added.

**Parameters:**

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

`class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshPyramidStairsTerrainCfg`

Bases: `SubTerrainBaseCfg`

[\[source\]](#)

Configuration for a pyramid stair mesh terrain.

Attributes:

<code>border_width</code>	The width of the border around the terrain (in m).
<code>step_height_range</code>	The minimum and maximum height of the steps (in m).
<code>step_width</code>	The width of the steps (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>holes</code>	If True, the terrain will have holes in the steps.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`border_width`: *float*

The width of the border around the terrain (in m). Defaults to 0.0.

The border is a flat terrain with the same height as the terrain.

`step_height_range`: *tuple[[float](#), [float](#)]*

The minimum and maximum height of the steps (in m).

step_width: *float*

The width of the steps (in m).

platform_width: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

holes: *bool*

If True, the terrain will have holes in the steps. Defaults to False.

If `holes` is True, the terrain will have pyramid stairs of length or width `platform_width` (depending on the direction) with no steps in the remaining area. Additionally, no border will be added.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[float, float]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacclab.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Inverted pyramid terrain

`isaacclab.terrains.trimesh.mesh_terrains.inverted_pyramid_stairs_terrain(difficulty:`

float, *cfg*: `mesh_terrains_cfg.MeshInvertedPyramidStairsTerrainCfg`) →

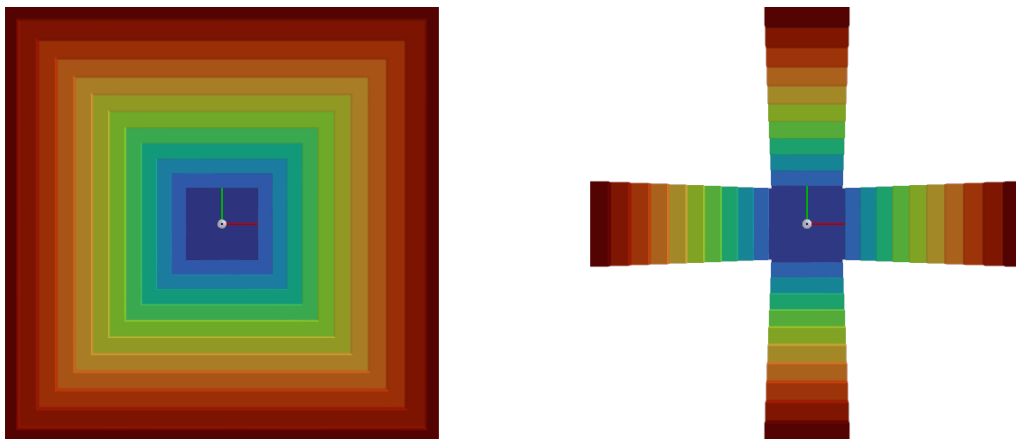
`tuple[list[trimesh.Trimesh], np.ndarray]`

[\[source\]](#)

Generate a terrain with a inverted pyramid stair pattern.

The terrain is an inverted pyramid stair pattern which trims to a flat platform at the center of the terrain.

If `cfg.holes` is True, the terrain will have pyramid stairs of length or width `cfg.platform_width` (depending on the direction) with no steps in the remaining area. Additionally, no border will be added.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

class

`isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshInvertedPyramidStairsTerrainCfg`

Bases: `MeshPyramidStairsTerrainCfg`

[\[source\]](#)

Configuration for an inverted pyramid stair mesh terrain.

Note

This is the same as `MeshPyramidStairsTerrainCfg` except that the steps are inverted.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>border_width</code>	The width of the border around the terrain (in m).
<code>step_height_range</code>	The minimum and maximum height of the steps (in m).
<code>step_width</code>	The width of the steps (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>holes</code>	If True, the terrain will have holes in the steps.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[*float*, *float*]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacclab.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

border_width: *float*

The width of the border around the terrain (in m). Defaults to 0.0.

The border is a flat terrain with the same height as the terrain.

step_height_range: *tuple[*float*, *float*]*

The minimum and maximum height of the steps (in m).

step_width: *float*

The width of the steps (in m).

platform_width: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

holes: *bool*

If True, the terrain will have holes in the steps. Defaults to False.

If `holes` is True, the terrain will have pyramid stairs of length or width `platform_width` (depending on the direction) with no steps in the remaining area. Additionally, no border will be added.

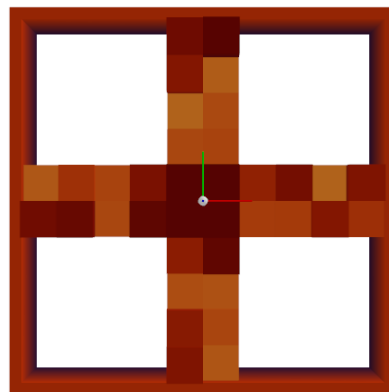
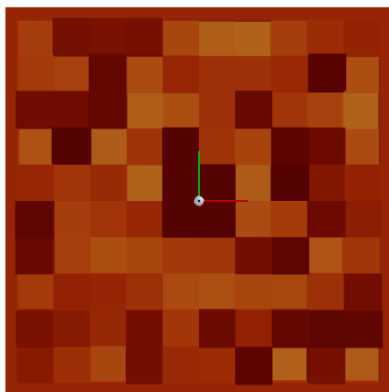
Random grid terrain

`isaacsim.terrains.trimesh.mesh_terrains.random_grid_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshRandomGridTerrainCfg)` → `tuple[list[trimesh.Trimesh], np.ndarray]` [\[source\]](#)

Generate a terrain with cells of random heights and fixed width.

The terrain is generated in the x-y plane and has a height of 1.0. It is then divided into a grid of the specified size `cfg.grid_width`. Each grid cell is then randomly shifted in the z-direction by a value uniformly sampled between `cfg.grid_height_range`. At the center of the terrain, a platform of the specified width `cfg.platform_width` is generated.

If `cfg.holes` is True, the terrain will have randomized grid cells only along the plane extending from the platform (like a plus sign). The remaining area remains empty and no border will be added.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

Raises:

- **ValueError** – If the terrain is not square. This method only supports square terrains.
- **RuntimeError** – If the grid width is large such that the border width is negative.

```
class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshRandomGridTerrainCfg [source]
```

Bases: `SubTerrainBaseCfg`

Configuration for a random grid mesh terrain.

Attributes:

<code>grid_width</code>	The width of the grid cells (in m).
<code>grid_height_range</code>	The minimum and maximum height of the grid cells (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>holes</code>	If True, the terrain will have holes in the steps.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`grid_width`: float

The width of the grid cells (in m).

`grid_height_range`: tuple[float, float]

The minimum and maximum height of the grid cells (in m).

`platform_width`: float

The width of the square platform at the center of the terrain. Defaults to 1.0.

`holes`: bool

If True, the terrain will have holes in the steps. Defaults to False.

If `holes` is True, the terrain will have randomized grid cells only along the plane extending from the

platform (like a plus sign). The remaining area remains empty and no border will be added.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[[float](#), [float](#)]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Rails terrain

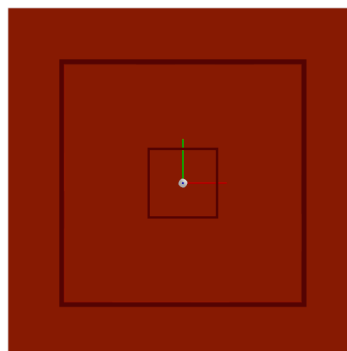
`isaacsim.terrains.trimesh.mesh_terrains.rails_terrain(difficulty: float, cfg:`

`mesh_terrains_cfg.MeshRailsTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]`

Generate a terrain with box rails as extrusions.

[\[source\]](#)

The terrain contains two sets of box rails created as extrusions. The first set (inner rails) is extruded from the platform at the center of the terrain, and the second set is extruded between the first set of rails and the terrain border. Each set of rails is extruded to the same height.



Parameters:

- **difficulty** – The difficulty of the terrain. this is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

`class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshRailsTerrainCfg` [\[source\]](#)

Bases: `SubTerrainBaseCfg`

Configuration for a terrain with box rails as extrusions.

Attributes:

<code>rail_thickness_range</code>	The thickness of the inner and outer rails (in m).
<code>rail_height_range</code>	The minimum and maximum height of the rails (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`rail_thickness_range: tuple[float, float]`

The thickness of the inner and outer rails (in m).

`rail_height_range: tuple[float, float]`

The minimum and maximum height of the rails (in m).

`platform_width: float`

The width of the square platform at the center of the terrain. Defaults to 1.0.

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: `tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by

`isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: `dict[str, FlatPatchSamplingCfg] | None`

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

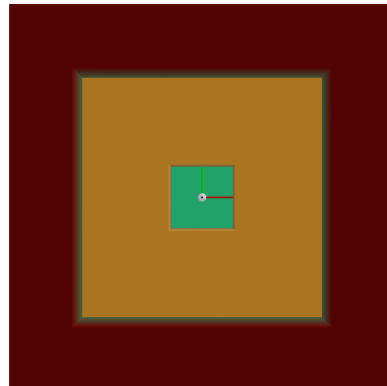
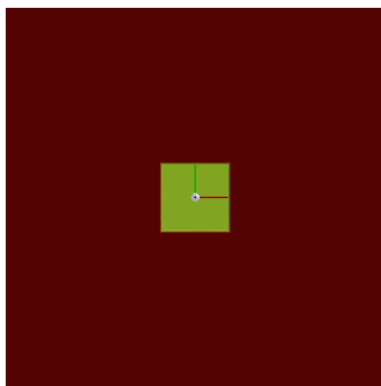
Pit terrain

`isaacsim.terrains.trimesh.mesh_terrains.pit_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshPitTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]`

Generate a terrain with a pit with levels (stairs) leading out of the pit.

[\[source\]](#)

The terrain contains a platform at the center and a staircase leading out of the pit. The staircase is a series of steps that are aligned along the x- and y- axis. The steps are created by extruding a ring along the x- and y- axis. If `is_double_pit` is True, the pit contains two levels.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

`class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshPitTerrainCfg`

[\[source\]](#)

Bases: `SubTerrainBaseCfg`

Configuration for a terrain with a pit that leads out of the pit.

Attributes:

<code>pit_depth_range</code>	The minimum and maximum height of the pit (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>double_pit</code>	If True, the pit contains two levels of stairs.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`pit_depth_range`: *tuple[float, float]*

The minimum and maximum height of the pit (in m).

`platform_width`: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

`double_pit`: *bool*

If True, the pit contains two levels of stairs. Defaults to False.

`proportion`: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size`: *tuple[float, float]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling`: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Box terrain

```
isaacsim.terrains.trimesh.mesh_terrains.box_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshBoxTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]
```

Generate a terrain with boxes (similar to a pyramid). [\[source\]](#)

The terrain has a ground with boxes on top of it that are stacked on top of each other. The boxes are created by extruding a rectangle along the z-axis. If `double_box` is True, then two boxes of height `box_height` are stacked on top of each other.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

```
class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshBoxTerrainCfg \[source\]
```

Bases: `SubTerrainBaseCfg`

Configuration for a terrain with boxes (similar to a pyramid).

Attributes:

<code>box_height_range</code>	The minimum and maximum height of the box (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>double_box</code>	If True, the pit contains two levels of stairs/boxes.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`box_height_range`: *tuple*[*float*, *float*]

The minimum and maximum height of the box (in m).

`platform_width`: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

`double_box`: *bool*

If True, the pit contains two levels of stairs/boxes. Defaults to False.

`proportion`: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size`: *tuple*[*float*, *float*]

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by

`isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling`: *dict*[*str*, *FlatPatchSamplingCfg*] | *None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

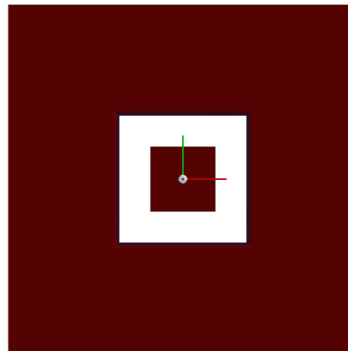
Gap terrain

```
isaacsim.terrains.trimesh.mesh_terrains.gap_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshGapTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]
```

Generate a terrain with a gap around the platform.

[\[source\]](#)

The terrain has a ground with a platform in the middle. The platform is surrounded by a gap of width `gap_width` on all sides.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

```
class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshGapTerrainCfg
```

[\[source\]](#)

Bases: `SubTerrainBaseCfg`

Configuration for a terrain with a gap around the platform.

Attributes:

<code>gap_width_range</code>	The minimum and maximum width of the gap (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

gap_width_range: `tuple[float, float]`

The minimum and maximum width of the gap (in m).

platform_width: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[[float](#), [float](#)]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by

`isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, [FlatPatchSamplingCfg](#)] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

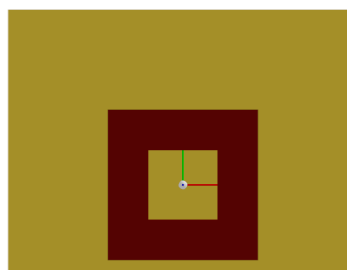
The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Floating ring terrain

`isaacsim.terrains.trimesh.mesh_terrains.floating_ring_terrain(difficulty: float,
cfg: mesh_terrains_cfg.MeshFloatingRingTerrainCfg) → tuple[list[trimesh.Trimesh],
np.ndarray]` [\[source\]](#)

Generate a terrain with a floating square ring.

The terrain has a ground with a floating ring in the middle. The ring extends from the center from `platform\_width` to `platform\_width + ring\_width` in the x and y directions. The thickness of the ring is `ring\_thickness` and the height of the ring from the terrain is `ring\_height`.



**Parameters:**

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

```
class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshFloatingRingTerrainCfg
```

Bases: `SubTerrainBaseCfg`

[\[source\]](#)

Configuration for a terrain with a floating ring around the center.

Attributes:

<code>ring_width_range</code>	The minimum and maximum width of the ring (in m).
<code>ring_height_range</code>	The minimum and maximum height of the ring (in m).
<code>ring_thickness</code>	The thickness (along z) of the ring (in m).
<code>platform_width</code>	The width of the square platform at the center of the terrain.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`ring_width_range`: *tuple[float, float]*

The minimum and maximum width of the ring (in m).

`ring_height_range`: *tuple[float, float]*

The minimum and maximum height of the ring (in m).

`ring_thickness`: *float*

The thickness (along z) of the ring (in m).

`platform_width`: *float*

The width of the square platform at the center of the terrain. Defaults to 1.0.

proportion: *float*

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: *tuple[*float*, *float*]*

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by

`isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: *dict[str, FlatPatchSamplingCfg] | None*

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

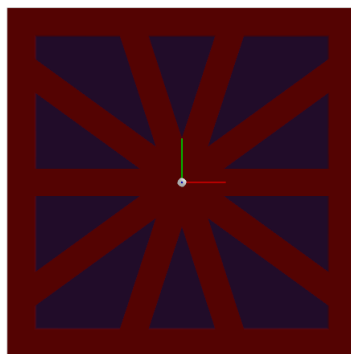
Star terrain

`isaacsim.terrains.trimesh.mesh_terrains.star_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshStarTerrainCfg)` → `tuple[list[trimesh.Trimesh], np.ndarray]`

Generate a terrain with a star.

[\[source\]](#)

The terrain has a ground with a cylinder in the middle. The star is made of `num_bars` bars with a width of `bar_width` and a height of `bar_height`. The bars are evenly spaced around the cylinder and connect to the peripheral of the terrain.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

Raises:

ValueError – If `num_bars` is less than 2.

`class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshStarTerrainCfg` [\[source\]](#)

Bases: `SubTerrainBaseCfg`

Configuration for a terrain with a star pattern.

Attributes:

<code>num_bars</code>	The number of bars per-side the star.
<code>bar_width_range</code>	The minimum and maximum width of the bars in the star (in m).
<code>bar_height_range</code>	The minimum and maximum height of the bars in the star (in m).
<code>platform_width</code>	The width of the cylindrical platform at the center of the terrain.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.

`num_bars: int`

The number of bars per-side the star. Must be greater than 2.

`bar_width_range: tuple[float, float]`

The minimum and maximum width of the bars in the star (in m).

`bar_height_range: tuple[float, float]`

The minimum and maximum height of the bars in the star (in m).

`platform_width: float`

The width of the cylindrical platform at the center of the terrain. Defaults to 1.0.

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

size: `tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

flat_patch_sampling: `dict[str, FlatPatchSamplingCfg] | None`

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

Repeated Objects Terrain

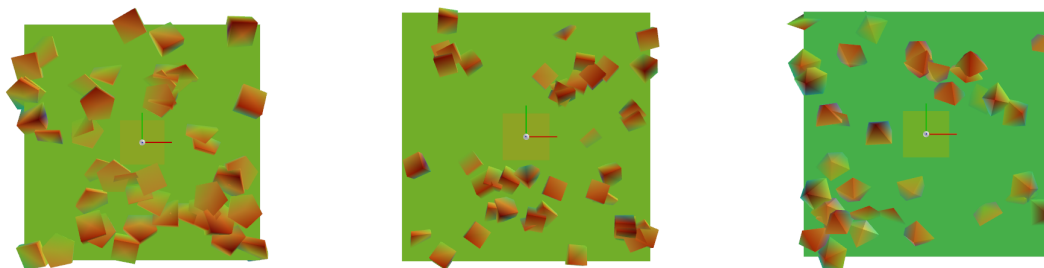
`isaacsim.terrains.trimesh.mesh_terrains.repeated_objects_terrain(difficulty: float, cfg: mesh_terrains_cfg.MeshRepeatedObjectsTerrainCfg) → tuple[list[trimesh.Trimesh], np.ndarray]` [\[source\]](#)

Generate a terrain with a set of repeated objects.

The terrain has a ground with a platform in the middle. The objects are randomly placed on the terrain s.t. they do not overlap with the platform.

Depending on the object type, the objects are generated with different parameters. The objects The types of objects that can be generated are: `"cylinder"`, `"box"`, `"cone"`.

The object parameters are specified in the configuration as curriculum parameters. The difficulty is used to linearly interpolate between the minimum and maximum values of the parameters.



Parameters:

- **difficulty** – The difficulty of the terrain. This is a value between 0 and 1.
- **cfg** – The configuration for the terrain.

Returns:

A tuple containing the tri-mesh of the terrain and the origin of the terrain (in m).

Raises:

ValueError – If the object type is not supported. It must be either a string or a callable.

```
class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshRepeatedObjectsTerrainCfg
```

Bases: `SubTerrainBaseCfg`

[\[source\]](#)

Base configuration for a terrain with repeated objects.

Classes:

<code>ObjectCfg</code>	Configuration of repeated objects.
------------------------	------------------------------------

Attributes:

<code>object_type</code>	The type of object to generate.
<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>object_params_start</code>	The object curriculum parameters at the start of the curriculum.
<code>object_params_end</code>	The object curriculum parameters at the end of the curriculum.
<code>max_height_noise</code>	"This parameter is deprecated, but stated here to support backward compatibility
<code>abs_height_noise</code>	The minimum and maximum amount of additive noise for the height of the objects.
<code>rel_height_noise</code>	The minimum and maximum amount of multiplicative noise for the height of the objects.
<code>platform_width</code>	The width of the cylindrical platform at the center of the terrain.
<code>platform_height</code>	The height of the platform.

```
class ObjectCfg
```

[\[source\]](#)

Bases: `object`

Configuration of repeated objects.

Attributes:

<code>num_objects</code>	The number of objects to add to the terrain.
<code>height</code>	The height (along z) of the object (in m).

`num_objects: int`

The number of objects to add to the terrain.

`height: float`

The height (along z) of the object (in m).

`object_type: Literal['cylinder', 'box', 'cone'] | callable`

The type of object to generate.

The type can be a string or a callable. If it is a string, the function will look for a function called

`make_{object_type}` in the current module scope. If it is a callable, the function will use the callable to generate the object.**`proportion: float`**

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size: tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by`isaacclab.scene.TerrainImporterCfg.size` attribute.**`flat_patch_sampling: dict[str, FlatPatchSamplingCfg] | None`**

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the corresponding configurations.

object_params_start: *ObjectCfg*

The object curriculum parameters at the start of the curriculum.

object_params_end: *ObjectCfg*

The object curriculum parameters at the end of the curriculum.

max_height_noise: *float* | *None*

“This parameter is deprecated, but stated here to support backward compatibility

abs_height_noise: *tuple[float, float]*

The minimum and maximum amount of additive noise for the height of the objects. Default is set to 0.0, which is no noise.

rel_height_noise: *tuple[float, float]*

The minimum and maximum amount of multiplicative noise for the height of the objects. Default is set to 1.0, which is no noise.

platform_width: *float*

The width of the cylindrical platform at the center of the terrain. Defaults to 1.0.

platform_height: *float*

The height of the platform. Defaults to -1.0.

If the value is negative, the height is the same as the object height.

class `isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshRepeatedPyramidsTerrainCfg`

Bases: `MeshRepeatedObjectsTerrainCfg`

[\[source\]](#)

Configuration for a terrain with repeated pyramids.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>max_height_noise</code>	"This parameter is deprecated, but stated here to support backward compatibility"
<code>abs_height_noise</code>	The minimum and maximum amount of additive noise for the height of the objects.
<code>rel_height_noise</code>	The minimum and maximum amount of multiplicative noise for the height of the objects.
<code>platform_width</code>	The width of the cylindrical platform at the center of the terrain.
<code>platform_height</code>	The height of the platform.
<code>object_type</code>	The type of object to generate.
<code>object_params_start</code>	The object curriculum parameters at the start of the curriculum.
<code>object_params_end</code>	The object curriculum parameters at the end of the curriculum.

Classes:

<code>ObjectCfg</code>	Configuration for a curriculum of repeated pyramids.
------------------------	--

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size: tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling: dict[str, FlatPatchSamplingCfg] | None`

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the

corresponding configurations.

max_height_noise: *float* | *None*

“This parameter is deprecated, but stated here to support backward compatibility

abs_height_noise: *tuple[float, float]*

The minimum and maximum amount of additive noise for the height of the objects. Default is set to 0.0, which is no noise.

rel_height_noise: *tuple[float, float]*

The minimum and maximum amount of multiplicative noise for the height of the objects. Default is set to 1.0, which is no noise.

platform_width: *float*

The width of the cylindrical platform at the center of the terrain. Defaults to 1.0.

platform_height: *float*

The height of the platform. Defaults to -1.0.

If the value is negative, the height is the same as the object height.

class ObjectCfg

[\[source\]](#)

Bases: `ObjectCfg`

Configuration for a curriculum of repeated pyramids.

Attributes:

<code>num_objects</code>	The number of objects to add to the terrain.
<code>height</code>	The height (along z) of the object (in m).
<code>radius</code>	The radius of the pyramids (in m).
<code>max_yx_angle</code>	The maximum angle along the y and x axis.
<code>degrees</code>	Whether the angle is in degrees.

num_objects: *int*

The number of objects to add to the terrain.

height: *float*

The height (along z) of the object (in m).

radius: *float*

The radius of the pyramids (in m).

max_yx_angle: *float*

The maximum angle along the y and x axis. Defaults to 0.0.

degrees: *bool*

Whether the angle is in degrees. Defaults to True.

object_type: *Literal['cylinder', 'box', 'cone'] | callable*

The type of object to generate.

The type can be a string or a callable. If it is a string, the function will look for a function called

`make_{object_type}` in the current module scope. If it is a callable, the function will use the callable to generate the object.

object_params_start: *ObjectCfg*

The object curriculum parameters at the start of the curriculum.

object_params_end: *ObjectCfg*

The object curriculum parameters at the end of the curriculum.

class `isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshRepeatedBoxesTerrainCfg`

Bases: `MeshRepeatedObjectsTerrainCfg`

[\[source\]](#)

Configuration for a terrain with repeated boxes.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>max_height_noise</code>	"This parameter is deprecated, but stated here to support backward compatibility"
<code>abs_height_noise</code>	The minimum and maximum amount of additive noise for the height of the objects.
<code>rel_height_noise</code>	The minimum and maximum amount of multiplicative noise for the height of the objects.
<code>platform_width</code>	The width of the cylindrical platform at the center of the terrain.
<code>platform_height</code>	The height of the platform.
<code>object_type</code>	The type of object to generate.
<code>object_params_start</code>	The box curriculum parameters at the start of the curriculum.
<code>object_params_end</code>	The box curriculum parameters at the end of the curriculum.

Classes:

<code>ObjectCfg</code>	Configuration for repeated boxes.
------------------------	-----------------------------------

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size: tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling: dict[str, FlatPatchSamplingCfg] | None`

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the

corresponding configurations.

max_height_noise: *float* | *None*

“This parameter is deprecated, but stated here to support backward compatibility

abs_height_noise: *tuple[float, float]*

The minimum and maximum amount of additive noise for the height of the objects. Default is set to 0.0, which is no noise.

rel_height_noise: *tuple[float, float]*

The minimum and maximum amount of multiplicative noise for the height of the objects. Default is set to 1.0, which is no noise.

platform_width: *float*

The width of the cylindrical platform at the center of the terrain. Defaults to 1.0.

platform_height: *float*

The height of the platform. Defaults to -1.0.

If the value is negative, the height is the same as the object height.

class ObjectCfg

[\[source\]](#)

Bases: `ObjectCfg`

Configuration for repeated boxes.

Attributes:

<code>num_objects</code>	The number of objects to add to the terrain.
<code>height</code>	The height (along z) of the object (in m).
<code>size</code>	The width (along x) and length (along y) of the box (in m).
<code>max_yx_angle</code>	The maximum angle along the y and x axis.
<code>degrees</code>	Whether the angle is in degrees.

num_objects: *int*

The number of objects to add to the terrain.

height: *float*

The height (along z) of the object (in m).

size: *tuple[float, float]*

The width (along x) and length (along y) of the box (in m).

max_yx_angle: *float*

The maximum angle along the y and x axis. Defaults to 0.0.

degrees: *bool*

Whether the angle is in degrees. Defaults to True.

object_type: *Literal['cylinder', 'box', 'cone'] | callable*

The type of object to generate.

The type can be a string or a callable. If it is a string, the function will look for a function called

`make_{object_type}` in the current module scope. If it is a callable, the function will use the callable to generate the object.

object_params_start: *ObjectCfg*

The box curriculum parameters at the start of the curriculum.

object_params_end: *ObjectCfg*

The box curriculum parameters at the end of the curriculum.

class isaacsim.terrains.trimesh.mesh_terrains_cfg.MeshRepeatedCylindersTerrainCfg

Bases: `MeshRepeatedObjectsTerrainCfg`

[\[source\]](#)

Configuration for a terrain with repeated cylinders.

Attributes:

<code>proportion</code>	Proportion of the terrain to generate.
<code>size</code>	The width (along x) and length (along y) of the terrain (in m).
<code>flat_patch_sampling</code>	Dictionary of configurations for sampling flat patches on the sub-terrain.
<code>max_height_noise</code>	"This parameter is deprecated, but stated here to support backward compatibility"
<code>abs_height_noise</code>	The minimum and maximum amount of additive noise for the height of the objects.
<code>rel_height_noise</code>	The minimum and maximum amount of multiplicative noise for the height of the objects.
<code>platform_width</code>	The width of the cylindrical platform at the center of the terrain.
<code>platform_height</code>	The height of the platform.
<code>object_type</code>	The type of object to generate.
<code>object_params_start</code>	The box curriculum parameters at the start of the curriculum.
<code>object_params_end</code>	The box curriculum parameters at the end of the curriculum.

Classes:

<code>ObjectCfg</code>	Configuration for repeated cylinder.
------------------------	--------------------------------------

`proportion: float`

Proportion of the terrain to generate. Defaults to 1.0.

This is used to generate a mix of terrains. The proportion corresponds to the probability of sampling the particular terrain. For example, if there are two terrains, A and B, with proportions 0.3 and 0.7, respectively, then the probability of sampling terrain A is 0.3 and the probability of sampling terrain B is 0.7.

`size: tuple[float, float]`

The width (along x) and length (along y) of the terrain (in m). Defaults to (10.0, 10.0).

In case the `TerrainImporterCfg` is used, this parameter gets overridden by `isaacsim.scene.TerrainImporterCfg.size` attribute.

`flat_patch_sampling: dict[str, FlatPatchSamplingCfg] | None`

Dictionary of configurations for sampling flat patches on the sub-terrain. Defaults to None, in which case no flat patch sampling is performed.

The keys correspond to the name of the flat patch sampling configuration and the values are the

corresponding configurations.

max_height_noise: *float* | *None*

“This parameter is deprecated, but stated here to support backward compatibility

abs_height_noise: *tuple[float, float]*

The minimum and maximum amount of additive noise for the height of the objects. Default is set to 0.0, which is no noise.

rel_height_noise: *tuple[float, float]*

The minimum and maximum amount of multiplicative noise for the height of the objects. Default is set to 1.0, which is no noise.

platform_width: *float*

The width of the cylindrical platform at the center of the terrain. Defaults to 1.0.

platform_height: *float*

The height of the platform. Defaults to -1.0.

If the value is negative, the height is the same as the object height.

class ObjectCfg

[\[source\]](#)

Bases: `ObjectCfg`

Configuration for repeated cylinder.

Attributes:

<code>num_objects</code>	The number of objects to add to the terrain.
<code>height</code>	The height (along z) of the object (in m).
<code>radius</code>	The radius of the pyramids (in m).
<code>max_yx_angle</code>	The maximum angle along the y and x axis.
<code>degrees</code>	Whether the angle is in degrees.

num_objects: *int*

The number of objects to add to the terrain.

height: *float*

The height (along z) of the object (in m).

radius: *float*

The radius of the pyramids (in m).

max_yx_angle: *float*

The maximum angle along the y and x axis. Defaults to 0.0.

degrees: *bool*

Whether the angle is in degrees. Defaults to True.

object_type: *Literal['cylinder', 'box', 'cone'] | callable*

The type of object to generate.

The type can be a string or a callable. If it is a string, the function will look for a function called

`make_{object_type}` in the current module scope. If it is a callable, the function will use the callable to generate the object.**object_params_start:** *ObjectCfg*

The box curriculum parameters at the start of the curriculum.

object_params_end: *ObjectCfg*

The box curriculum parameters at the end of the curriculum.

Utilities

Functions:

<code>color_meshes_by_height</code> (meshes, **kwargs)	Color the vertices of a trimesh object based on the z-coordinate (height) of each vertex, using the Turbo colormap.
<code>create_prim_from_mesh</code> (prim_path, mesh, **kwargs)	Create a USD prim with mesh defined from vertices and triangles.
<code>find_flat_patches</code> (wp_mesh, num_patches, ...)	Finds flat patches of given radius in the input mesh.

```
isaacsim.terrains.utils.color_meshes_by_height(meshes: list[trimesh.Trimesh],
**kwargs) → trimesh.Trimesh \[source\]
```

Color the vertices of a trimesh object based on the z-coordinate (height) of each vertex, using the Turbo colormap. If the z-coordinates are all the same, the vertices will be colored with a single color.

Parameters:

meshes – A list of trimesh objects.

Keyword Arguments:

- **color** – A list of 3 integers in the range [0,255] representing the RGB color of the mesh. Used when the z-coordinates of all vertices are the same. Defaults to [172, 216, 230].
- **color_map** – The name of the color map to be used. Defaults to “turbo”.

Returns:

A trimesh object with the vertices colored based on the z-coordinate (height) of each vertex.

```
isaacsim.terrains.utils.create_prim_from_mesh(prim_path: str, mesh:
trimesh.Trimesh, **kwargs) \[source\]
```

Create a USD prim with mesh defined from vertices and triangles.

The function creates a USD prim with a mesh defined from vertices and triangles. It performs the following steps:

- Create a USD Xform prim at the path `{prim_path}`.
- Create a USD prim with a mesh defined from the input vertices and triangles at the path `{prim_path}/mesh`.
- Assign a physics material to the mesh at the path `{prim_path}/physicsMaterial`.
- Assign a visual material to the mesh at the path `{prim_path}/visualMaterial`.

Parameters:

- **prim_path** – The path to the primitive to be created.
- **mesh** – The mesh to be used for the primitive.

Keyword Arguments:

- **translation** – The translation of the terrain. Defaults to None.
- **orientation** – The orientation of the terrain. Defaults to None.
- **visual_material** – The visual material to apply. Defaults to None.
- **physics_material** – The physics material to apply. Defaults to None.

```
isaacsim.terrains.utils.find_flat_patches(wp_mesh: wp.Mesh, num_patches: int,
patch_radius: float | list[float], origin: np.ndarray | torch.Tensor | tuple[float,
float, float], x_range: tuple[float, float], y_range: tuple[float, float], z_range:
tuple[float, float], max_height_diff: float) → torch.Tensor \[source\]
```

Finds flat patches of given radius in the input mesh.

The function finds flat patches of given radius based on the search space defined by the input ranges. The search space is characterized by origin in the mesh frame, and the x, y, and z ranges. The x and y ranges are used to sample points in the 2D region around the origin, and the z range is used to filter patches based on the height of the points.

The function performs rejection sampling to find the patches based on the following steps:

1. Sample patch locations in the 2D region around the origin.
2. Define a ring of points around each patch location to query the height of the points using ray-casting.
3. Reject patches that are outside the z range or have a height difference that is too large.
4. Keep sampling until all patches are valid.

Parameters:

- **wp_mesh** – The warp mesh to find patches in.
- **num_patches** – The desired number of patches to find.
- **patch_radius** – The radii used to form patches. If a list is provided, multiple patch sizes are checked. This is useful to deal with holes or other artifacts in the mesh.
- **origin** – The origin defining the center of the search space. This is specified in the mesh frame.
- **x_range** – The range of X coordinates to sample from.
- **y_range** – The range of Y coordinates to sample from.
- **z_range** – The range of valid Z coordinates used for filtering patches.
- **max_height_diff** – The maximum allowable distance between the lowest and highest points on a patch to consider it as valid. If the difference is greater than this value, the patch is rejected.

Returns:

A tensor of shape (num_patches, 3) containing the flat patches. The patches are defined in the mesh frame.

Raises:

RuntimeError – If the function fails to find valid patches. This can happen if the input parameters are not suitable for finding valid patches and maximum number of iterations is reached.